



KMK2473 COMPUTER GRAPHICS

INDIVIDUAL ASSIGNMENT

Interactive 3D Solar System and Space Model

NAME	MOHAMAD FARIS IRSYAD BIN SOBLE
MATRIC NUMBER	105004
LECTURER'S NAME	PROF MADYA DR ABDULRAZAK YAHYA SALEH AL-HABABI
VIDEO LINK	https://youtu.be/S7znJck7RRc
DATE OF SUBMISSION	5 JUNE 2026

Introduction

The aim of this project is to present an interactive 3D Solar System model developed with WebGL and the three.js library, with the possibility of varying the angle of the camera, the speed of the animation, and the effects applied, through a GUI.

Concept Behind Computer Graphics

For each of these concepts, explain how your code illustrates them:

- **Geometry and Meshes:** The planets are created procedurally using `SphereGeometry`. This illustrates just how the 3D objects are created from mathematical vertices and faces.
- **Lighting Models:** The scene uses `MeshStandardMaterial` material, which has physical reaction to light. Discuss the concepts of diffuse and specular reflection, by casting shadows from a central `PointLight` (the Sun) and highlighting edges with a `DirectionalLight` (Rim Light).
- **Transformation Matrices:**
 - **Translation:** Planets move around in orbits in relation to a pivot point.
 - **Orbit:** Each planet orbits around the Sun (`mesh.rotation.x`).
 - **Scaling:** Planets get a pulsing scaling effect with a sine wave function.

Implementation Steps

Explain your code comments and the technical procedure:

1. Environment Setup: Initialized Scene, Camera and Renderer, which has the shadow mapping option turned on.
2. Procedural Planet Creation: Developed a reusable function `createPlanet()` to dynamically generate meshes, materials and orbit rings. This eliminates code duplication.
3. Instanced Lighting: Instanced Ambient Light for base visibility, Point light for the sun's light/shadows and Directional Light for rim lights.
4. Starfield Background: Using `BufferGeometry` and `PointsMaterial` to efficiently draw 2,000 stars without creating too many mesh objects.
5. Position Update: Used `requestAnimationFrame` to update positions frame by frame.
6. Interaction Logic: Added `MouseClicks` (toggle play/pause), `KeyboardArrows` (manual camera movement) event listeners.

Technical Features Used (Rubric Mapping)

- Multiple GUI Interfaces: Used `dat.GUI` for creating two folders (Speed, Auto-Rotate) and (Brightness, Shadows, Collision Toggle).
- Camera Movement: Camera automatically follows the sun as follows: `Math.sin` and `Math.cos` are used here. This can be changed by pressing the Arrow Keys.
- User Guide Manual: Manual about controls in HTML in the user interface (`#user-guide div`).
- Input Controls:
 - Click to pause or resume the animation.
 - The camera can be adjusted manually in the following ways: Keyboard: Arrow keys.
- Effects:
 - Shading: Realistic (`MeshStandardMaterial`).
 - Collision Detection: Logic Tests Distance between Earth and Mars. Their colors are changed to Red if < 2.5 units to emulate a warning/collision state.

Challenges and Solutions

- Challenge: To have the planets move properly around the center and on their own axis as well.
- I think you need to be using a “Pivot” object (THREE.Object3D). The pivot rotates and the planet is placed on the pivot. This is a way of separating orbital rotation from self-rotation.
- Challenge: How to implement collision detection of moving objects.
- Solution: I used the .distanceTo() method is used in the animation loop to continually calculate the distance between Earth and Mars vectors.

Conclusion

The final product of this assignment was a successful demonstration of WebGL capabilities and how to create an immersive 3D environment, including the use of math (trigonometry, vectors) to create visual graphics.

References

Three.js Documentation: <https://threejs.org/docs/>

dat.GUI Documentation: <https://github.com/dataarts/dat.gui>